ML OPS Advanced

A.SCHUTZ

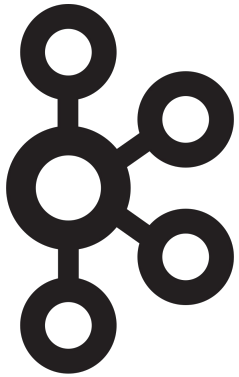
Docker-compose

BROKER

notion sur les brokers

Un broker permet de gérer la communication **asynchrone** entre services, ce qui évite de bloquer l'application en attendant une réponse, permettant ainsi une exécution plus fluide et réactive.

Pour des développeurs d'applications en IA, l'utilisation d'un broker est cruciale pour découpler les processus, exécuter des tâches parallèles et ne pas bloquer l'application pendant des calculs lourds ou des traitements de données.



Apache Kafka - Rabbit MQ - Celery

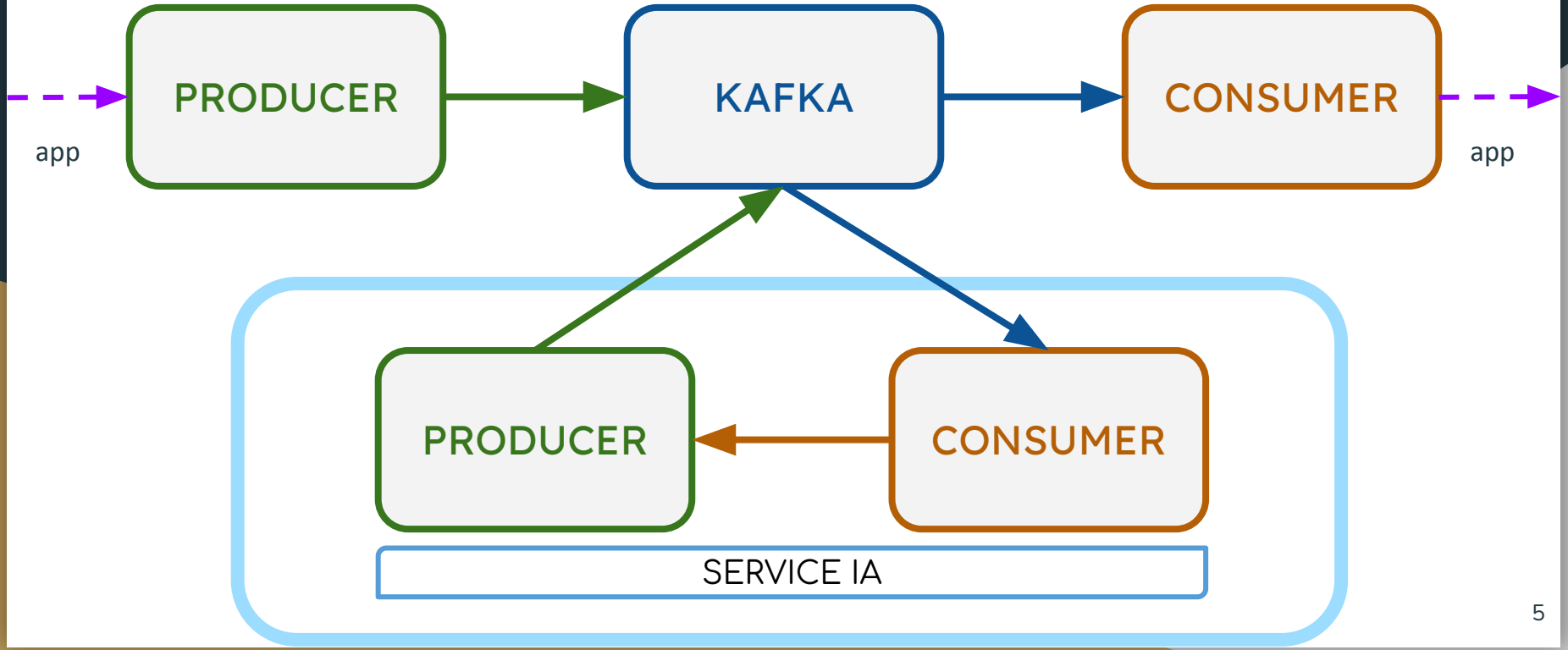
Kafka :

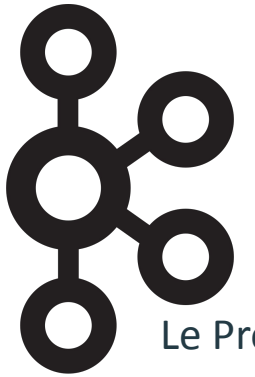
Kafka est une plateforme de streaming distribuée conçue pour gérer un grand volume de messages en temps réel, permettant une communication hautement scalable entre services.

Celery :

Celery est un framework permettant d'exécuter des tâches asynchrones ou planifiées, souvent utilisé pour des traitements lourds en arrière-plan. Il utilise le broker RabbitMQ entre autre

Au cœur d’une architecture de flux de données, Kafka sert de plateforme centrale, où les Producers publient des flux de données et les Consumers les consomment pour traitement en temps réel.





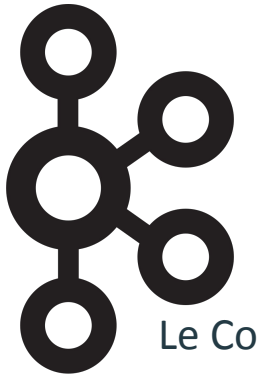
broker (kafka) - Producer (Producteur)

Le Producer c’est comme écrire des messages et les envoyer à une boîte aux lettres.

Exemple:

- Imaginez une application de réseau social. Chaque fois qu'un utilisateur poste un message, cette action est "produite".
- L'application (le Producer) prend le message, le transforme en un format que Kafka comprend, et l'envoie à Kafka.
- En gros le producer, "produit" des données, qui seront ensuite stockées dans Kafka.

Le Producer envoie des données à Kafka.



broker (kafka) - Consumer (Consommateur)

Le Consumer c’est comme récupérer les messages de la boîte aux lettres et les lire.

Exemple:

- Une autre application, qui analyse les messages du réseau social pour voir les tendances, est un Consumer.
- Elle se connecte à Kafka, demande les nouveaux messages, et les traite.
- En gros, le consommateur "consomme" les données qui sont stockées dans Kafka.

Le Consumer reçoit et traite les données de Kafka.

Producers:

- Ce sont les applications qui génèrent des données et les envoient à Kafka.
- Il peut y avoir plusieurs Producers envoyant des données à Kafka en même temps.

Kafka (Broker):

- C'est le cœur du système. Il stocke les messages envoyés par les Producers.
- Kafka organise les messages en "topics", qui sont comme des catégories de messages.

Consumers:

- Ce sont les applications qui lisent et traitent les données stockées dans Kafka.
- Comme les Producers, il peut y avoir plusieurs Consumers lisant les données de Kafka en même temps.
- Plusieurs Consumers peuvent lire les mêmes Topics en même temps, ou des Topics différents.

Topic

Un topic Kafka est comme une "catégorie" ou un "flux" nommé où les messages sont stockés.

Les producteurs envoient des messages à des topics spécifiques, et les consommateurs lisent les messages de ces mêmes topics.

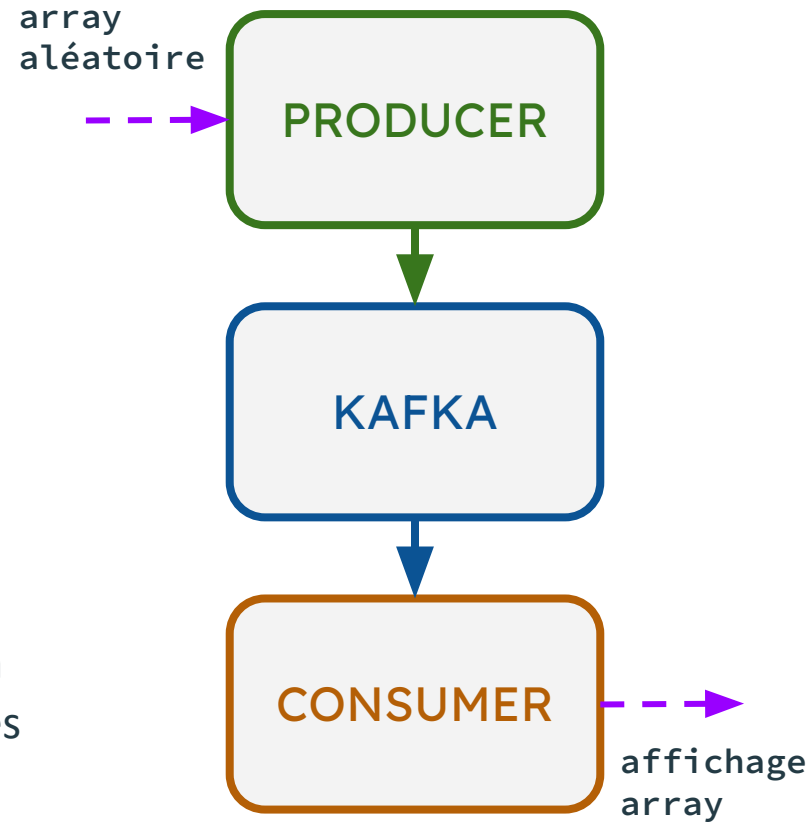
Les topics permettent d'organiser et de classer les flux de données, facilitant la gestion et le traitement de l'information.

Exemple

Une array aléatoire est généré toutes les secondes.

Le producer en fait un topic

Le consumer affiche en temps réels les données reçue




```
kafka-project/
├── producer/
│   ├── producer.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── wait-for-kafka.sh
│   └── .env
├── consumer/
│   ├── consumer.py
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── wait-for-kafka.sh
│   └── .env
├── docker-compose.yml
└── README.md
```

https://github.com/DeVerMyst/docker-compose-kafka.git

Celery 

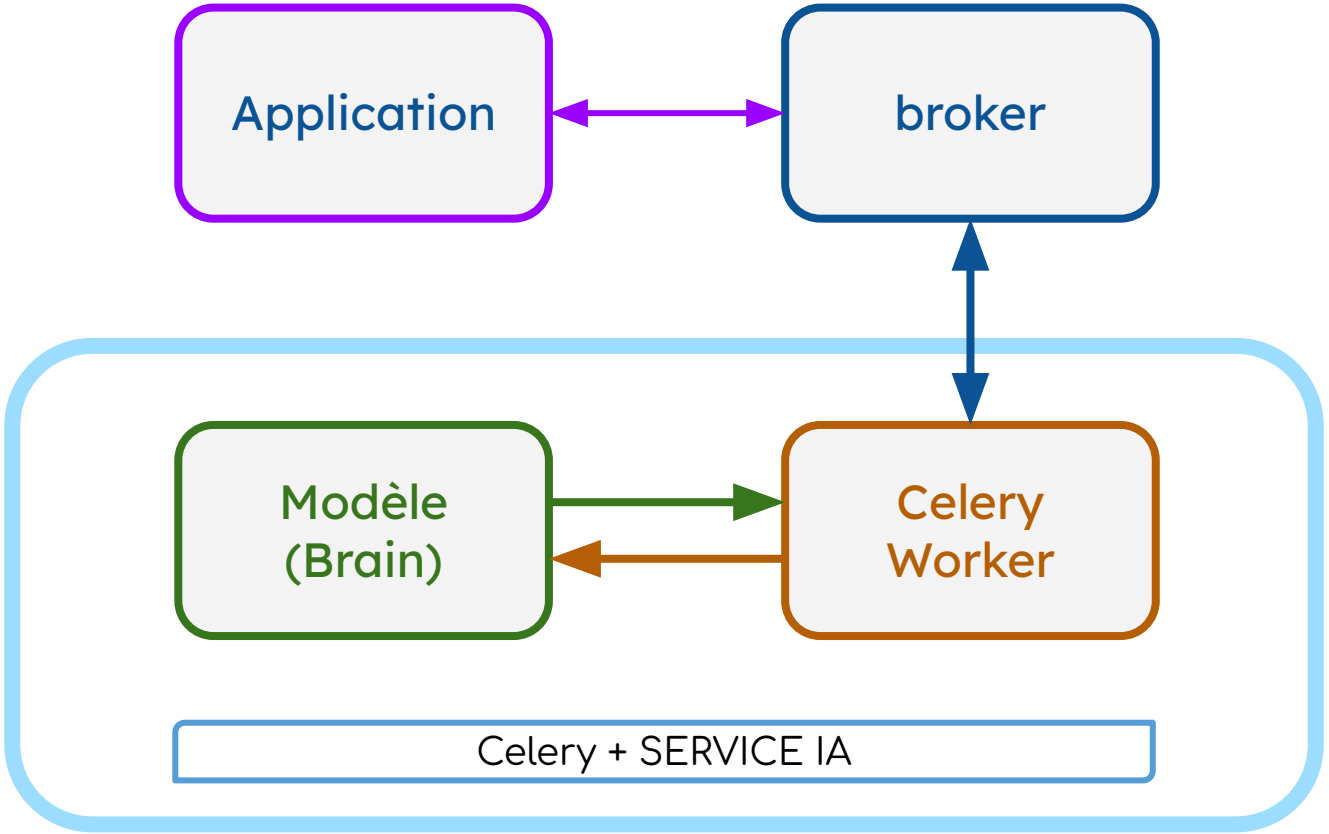
Système de file d'attente distribué permettant l'exécution asynchrone et la gestion de tâches lourdes en arrière-plan

→ c’est le moteur utilisé pour lancer des requêtes et que votre application continue de fonctionner sans attendre patiemment que les prédiction soit faites!
→ il ne fonctionne pas tout seul, il a besoin d’un système de gestion des données

Redis : Moteur de stockage de données en mémoire à ultra-haute performance, utilisé ici comme courtier de messages léger. Tout est dans la RAM mais il n’y a aucune vérification.

RabbitMQ : Agent de transfert de messages implémentant le protocole AMQP pour garantir une distribution robuste et sécurisée des données. Plus lourd mais vérifie tout, il est très utilisé en industrie.

Celery


Application
broker
Modèle
(Brain)
Celery
Worker
Celery + SERVICE IA

Broker

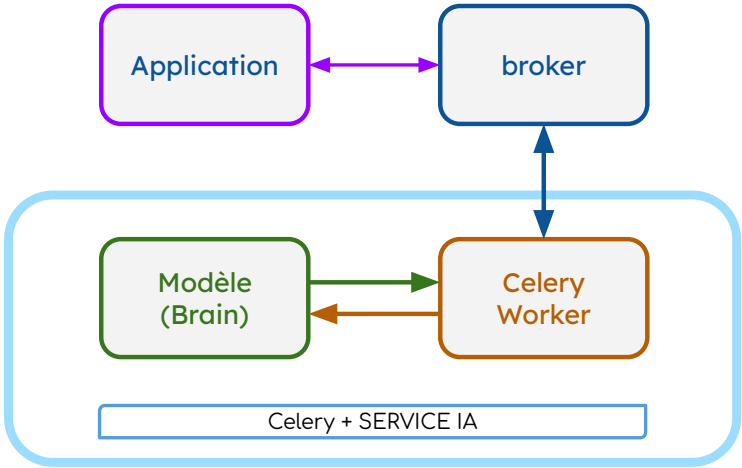
```yaml
redis:
  image: redis:alpine
  container_name : redis_broker
  ports:
    - "6379:6379"
  networks:
    - mlops-network
```

```yaml
  rabbitmq:
    image: rabbitmq:3-management-alpine
    container_name : rabbitmq_broker
    ports:
      - "15672:15672"  # Interface d'admin
RabbitMQ
    networks:
      - mlops-network
```

L'application envoie des requêtes dans le broker

Le worker Celery et dès qu'il est disponible récupère un travail à faire.

Quand le travail est terminé le résultat est renvoyé au broker et l'application n'a plus qu'à récupérer le résultat.



worker

```
celery -A worker.app worker --loglevel=info
```

`celery`

C'est l'exécutable principal. Il lance le framework Celery dans le container.

`-A worker.app`

Il dit à Celery où trouver l'instance de l'application worker.py. app = Celery(...).

`worker`

C'est la sous-commande qui indique à Celery que ce container doit agir comme un Worker

```
command: ["celery", "-A", "worker.app", "worker", "--loglevel=info", "--concurrency=1"]
```

```yaml
worker:
  build: .
  container_name : celery_worker
  command: celery -A worker.app worker --loglevel=info
  environment :
    - CELERY_BROKER_URL=amqp://guest:guest@rabbitmq:5672//
    - CELERY_RESULT_BACKEND=rpc://
    - MODEL_LATENCY=5
  depends_on :
    - rabbitmq
  networks :
    - ml-network
```

```yaml
worker:
  build: .
  container_name : celery_worker
  # Commande pour lancer le worker Celery
  command: celery -A worker.app worker --loglevel=info
  environment :
    - CELERY_BROKER_URL=redis://redis:6379/0
    - CELERY_RESULT_BACKEND=redis://redis:6379/0
    - MODEL_LATENCY=5
  depends_on :
    - redis
  networks :
    - ml-network
```

worker

C’est un peu comme une API, on déclare un app Celery et on utilise un décorateur nommé qui sera connu par l’API de fast_api.

Ici le prédicteur est bidon, il attend, normalement 5 secondes mais on dirait qu’il bug et ne trouve pas les 5 secondes à attendre.

Celery nomme les tâches en fonction de leur chemin, en forçant le nom on évite des problèmes.

Dans l’api on appelle pas `predict_task` mais on appelle `predict_task.delay()` car cela ajoute la tâche à la file d’attente.

```python
import os
import time

from celery import Celery

# Configuration dynamique via Docker Compose (avec défaut)
BROKER_URL = os.getenv("CELERY_BROKER_URL",
"redis://localhost:6379/0")
RESULT_BACKEND = os.getenv("CELERY_RESULT_BACKEND",
"redis://localhost:6379/0")

app = Celery("ml_tasks",
             broker=BROKER_URL,
             backend=RESULT_BACKEND)

@app.task(name="predict_task")
def predict_task():
    # Simulation d'un modèle d'IA lourd
    processing_time = int(os.getenv("MODEL_LATENCY", "5"))
    time.sleep(processing_time)
    return {"status": "complete", "prediction": 42}
```

API

Dans l'api on appelle pas `predict_task` mais on appelle `predict_task.delay()` car cela ajoute la tâche à la file d'attente.

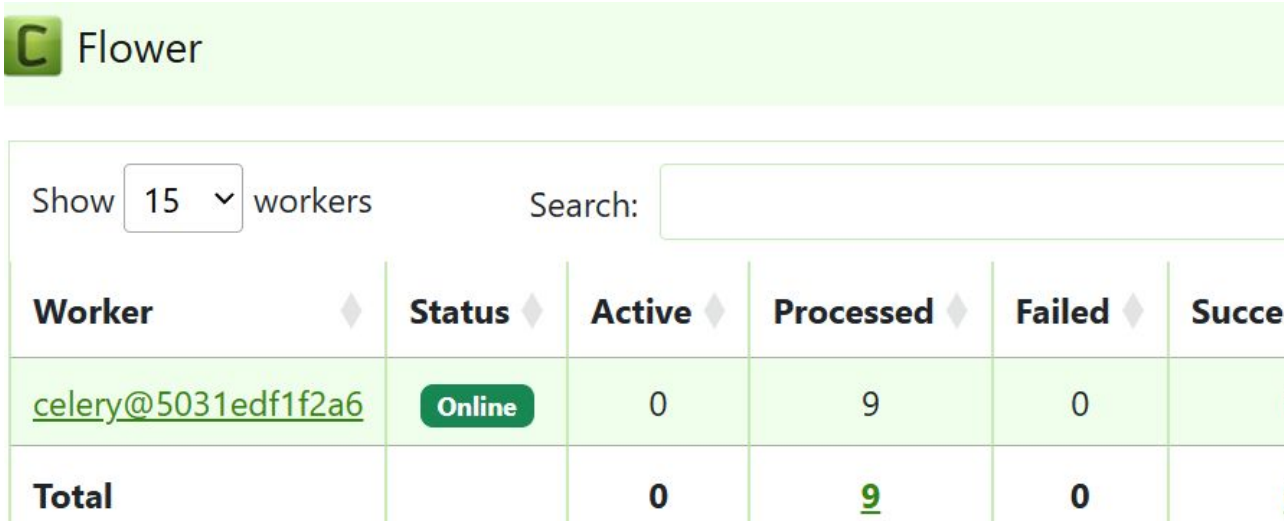
```python
from celery.result import AsyncResult
from fastapi import FastAPI

from worker import predict_task

app = FastAPI()


@app.post("/predict")
def run_prediction():
    # On envoie la tâche au broker sans attendre
    job = predict_task.delay()
    return {"task_id": job.id}
```

Flower


Flower
Show 15 workers
Search:
Worker
Status
Active
Processed
Failed
Succe
celery@5031edf1f2a6
Online
0
9
0
Total
0
9
0

Flower

Outil de monitoring en temps réel pour Celery, offrant une interface web pour surveiller l'exécution des tâches.

Permet de visualiser l'état des workers, d'annuler des tâches en cours et d'analyser les temps de réponse de vos modèles IA.

Il se connecte directement au Broker (Redis ou RabbitMQ) pour intercepter et afficher les événements du cluster Celery.

On Branche Flower sur le Broker.

```
flower:
    image: mher/flower:latest
    container_name : flower_monitor
    environment :
      # URL du Broker pour les tâches
      - CELERY_BROKER_URL=amqp://guest:guest@rabbitmq:5672//
      # URL de l'API RabbitMQ pour les stats avancées
      -
FLOWER_BROKER_API=http://guest:guest@rabbitmq:15672/api/
    ports:
      - "5555:5555"
    depends_on :
      - rabbitmq
      - worker
    networks :
      - ml-network
```

```
flower:
    image: mher/flower:latest
    container_name : flower_monitor
    environment :
      -
CELERY_BROKER_URL=redis://redis:6379/0
    ports:
      - "5555:5555"
    depends_on :
      - redis
      - worker
    networks :
      - ml-network
```

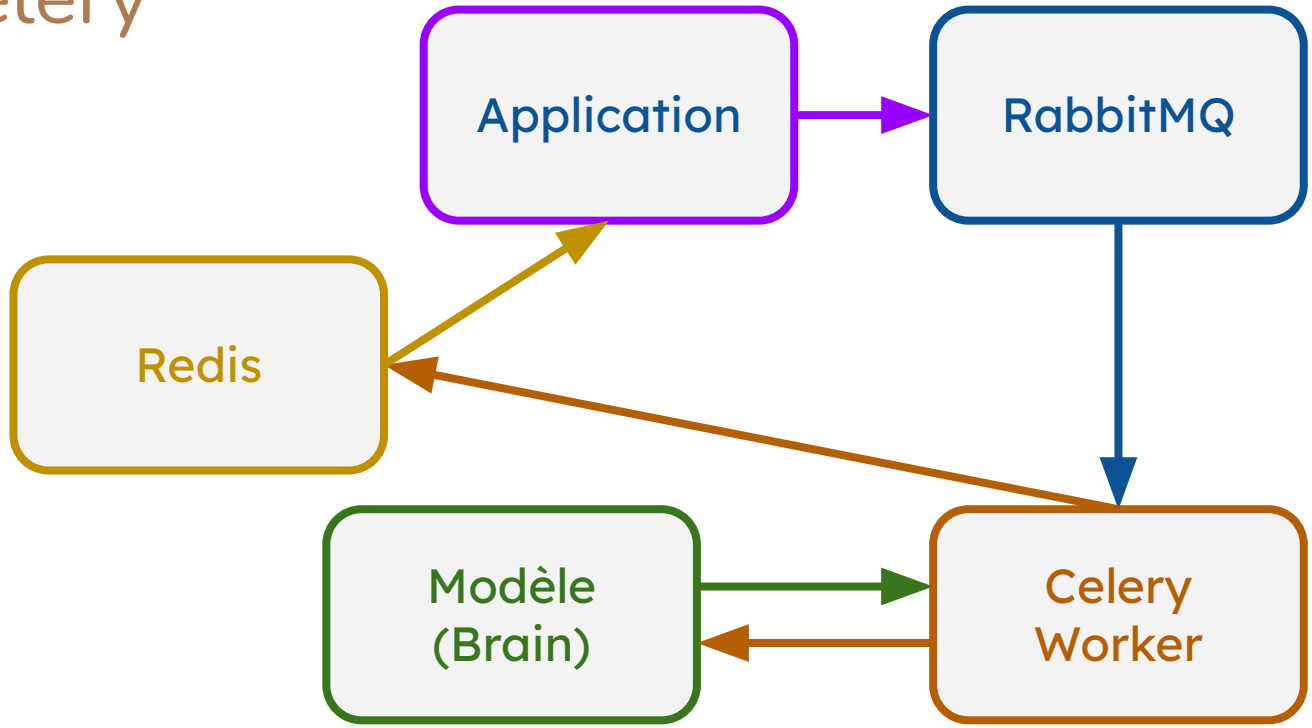
Celery - Redis et RabbitMQ

RabbitMQ est top pour envoyer les messages de façon sécurisé, il peut gérer des fils d'attente absolument énorme. Mais il a besoin de vite libérer la mémoire. Ce n'est pas une base de données.

Redis est top pour la gestion de la mémoire et il est super rapide.

On peut avoir le meilleur des deux mondes (standard de l'industrie) en utilisant RabbitMQ pour gérer les files d'attente et Redis pour le retour des données.

Celery



Celery + SERVICE IA

Celery - Scalabilité horizontale

Si on ne donne pas de nom au worker dans `docker-compose.final.yml` alors on peut augmenter le nombre de worker en une commande

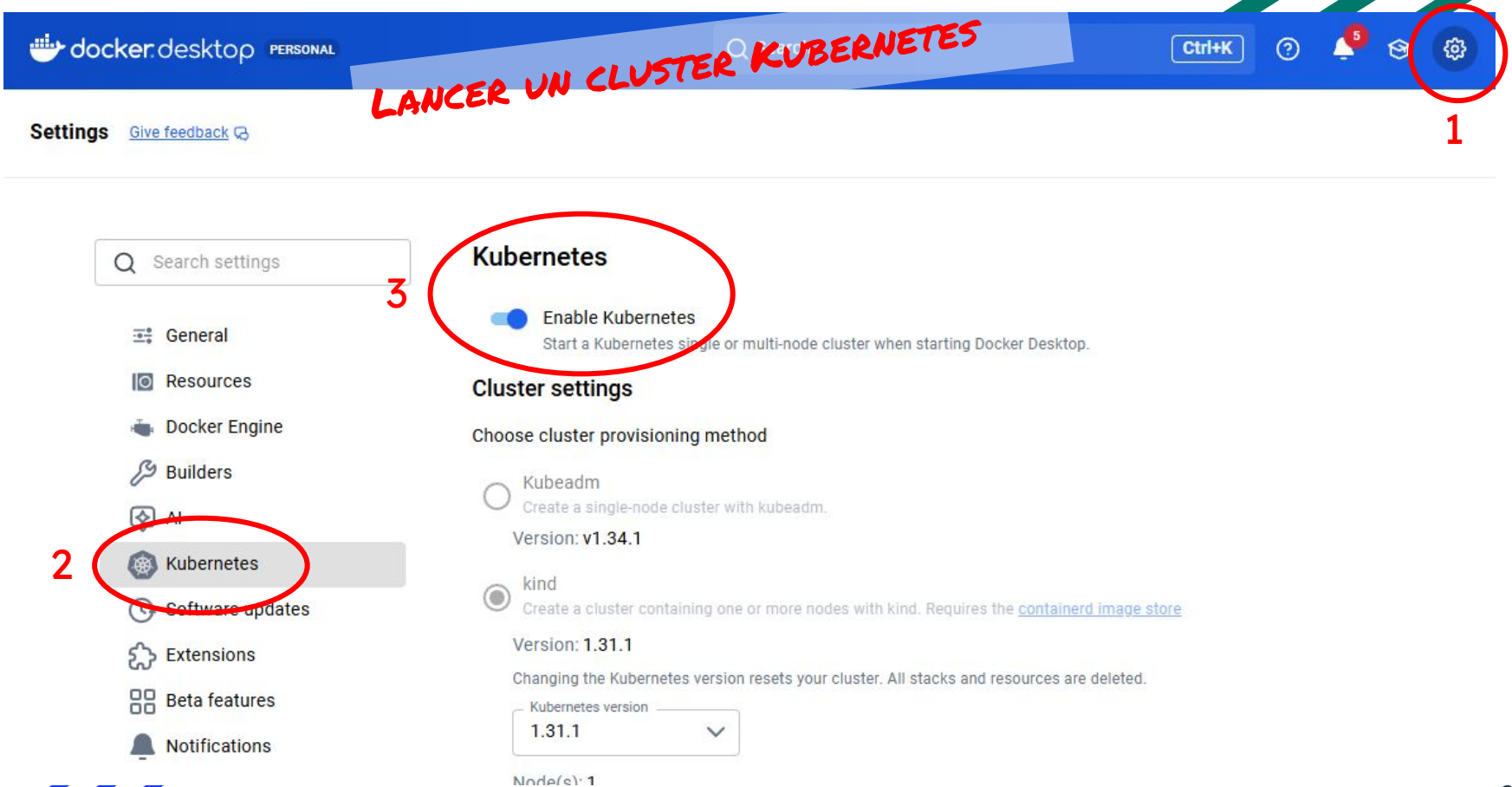
```
docker compose -f docker-compose.final.yml up --scale worker=3 -d
```

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load A
celery@06148eb669a2	Online	0	0	0	0	0	0.04,
celery@2bdb71b2f0ae	Online	0	0	0	0	0	0.04,
celery@ed61535b512f	Online	0	0	0	0	0	0.04,

De Docker-Compose vers Kubernetes

Introduction

LANCER UN CLUSTER KUBERNETES



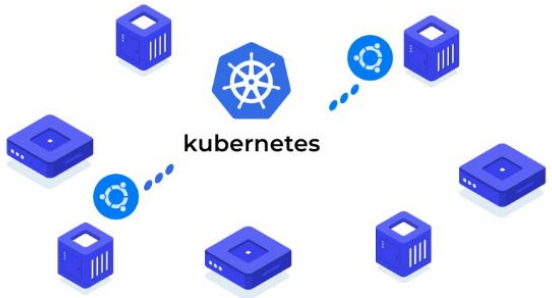
Docker Compose vs Kubernetes



Docker Compose coordonne des services sur un serveur unique. Cette approche limite les capacités de passage à l'échelle et de haute disponibilité, car tout repose sur une seule machine physique ou virtuelle.

Kubernetes orchestre des conteneurs à l'échelle d'un cluster Cloud. La technologie permet de piloter une flotte de serveurs tout en restant exploitable sur une machine unique pour les phases de développement ou les infrastructures légères.



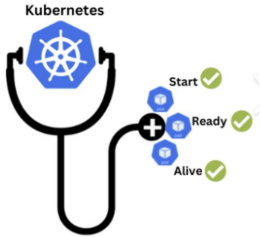


Kubernetes - HEALING

Kubernetes garantit la résilience du système par des mécanismes d'auto-réparation performants. En cas de panne d'un conteneur ou d'un nœud, l'orchestrateur détecte l'anomalie et recrée instantanément les instances manquantes pour maintenir l'état défini. Cette surveillance constante élimine les interventions manuelles en cas d'erreur logicielle ou matérielle.

Tuto + TP:

https://github.com/DeVerMyst/tuto_kubernetes/



Structure du projet

```
PROJET-MLOPS/
├── api                  # Code FastAPI
├── front                # Code Streamlit
├── worker               # Code Celery
├── Dockerfile           # Dockerfile
├── k8s/                 # Manifestes Kubernetes (Découpage)
│   ├── redis.yaml       # Déploiement + Service (Stockage)
│   ├── rabbitmq.yaml    # Déploiement + Service (Broker)
│   ├── api.yaml         # Déploiement + Service (Inférence)
│   ├── worker.yaml      # Déploiement (Calcul)
│   └── hpa.yaml         # Autoscaling du Worker
└── docker-compose.yml   # Conservé pour le dev local
```

Le passage de Docker Compose à Kubernetes impose une décomposition granulaire. L'arborescence évolue pour séparer la logique de construction (Docker) de la logique d'orchestration (Kubernetes).

L'organisation repose sur un dossier racine contenant les sources et un sous-dossier dédié aux manifestes Kubernetes (k8s). Cette séparation permet de versionner l'infrastructure indépendamment du code applicatif.

Chaque service du docker-compose.yml devient un fichier YAML et contient généralement deux objets :

- Le Déploiement : Remplace la section services et définit l'image, les ressources et les réplicas.
- Le Service : Remplace la résolution DNS interne de Docker. Sans lui, les conteneurs ne peuvent pas communiquer entre eux via un nom fixe.

Cette modularité permet de mettre à jour le Worker ou de modifier les règles du HPA sans risquer de perturber le fonctionnement de l'API ou de la base de données.

On peut aussi avoir 1 yaml par structure (monitoring, infra, code etc…) - avoir des sous dossiers etc…

Déploiement des manifestes



Pour lancer les services, on abandonne le docker compose up. L'interaction se fait désormais via l'outil en ligne de commande `kubectl`.

L'application des fichiers peut se faire de manière individuelle ou groupée. La commande apply est déclarative : elle crée la ressource si elle n'existe pas ou la met à jour si une modification est détectée dans le YAML.

- Lancer un service spécifique : `kubectl apply -f k8s/redis.yaml`
- Lancer toute l'infrastructure d'un coup : `kubectl apply -f k8s/`

```
PROJET-MLOPS/
├── api/
├── worker/
├── k8s/
│   ├── redis
```

Surveillance de l'état du cluster



Une fois les ordres envoyés, il est nécessaire de vérifier que Kubernetes parvient à instancier les ressources.
La commande get permet de lister les composants et d'observer leur statut.

- Lister les Pods (les containers) : `kubectl get pods`
- Lister les Services (les accès réseau) : `kubectl get svc`
- Surveiller en temps réel : `kubectl get pods -w`

utile pour voir les Workers apparaître lors du scaling.

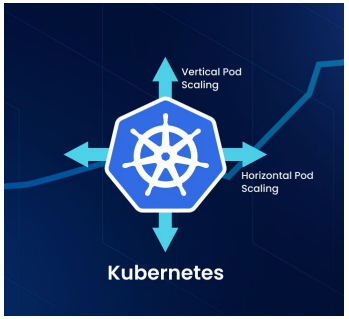
Analyse et Débogage



En cas d'erreur (`statut Error` ou `CrashLoopBackOff`), deux commandes sont indispensables pour comprendre l'origine du problème :

- Consulter les logs : `kubectl logs <nom-du-pod>`
- Inspecter les événements système : `kubectl describe pod <nom-du-pod>`

Kubernetes - HPA



Le scaling s'appuie sur une gestion dynamique des ressources. Le système ajuste le nombre de réplicas en fonction de la charge réelle, permettant d'absorber les pics de trafic sans dégradation de service. Cette agilité optimise l'utilisation des serveurs : l'infrastructure s'étend lors des phases de calcul intensif et se réduit automatiquement durant les périodes d'inactivité.

Le Horizontal Pod Autoscaler (HPA) automatise cette élasticité. Il surveille la consommation CPU ou RAM et ajuste instantanément le nombre de Workers. En cas de pic de prédictions, le système déploie de nouvelles unités ; dès le retour au calme, il supprime les instances superflues pour optimiser les coûts et les ressources.

Points importants

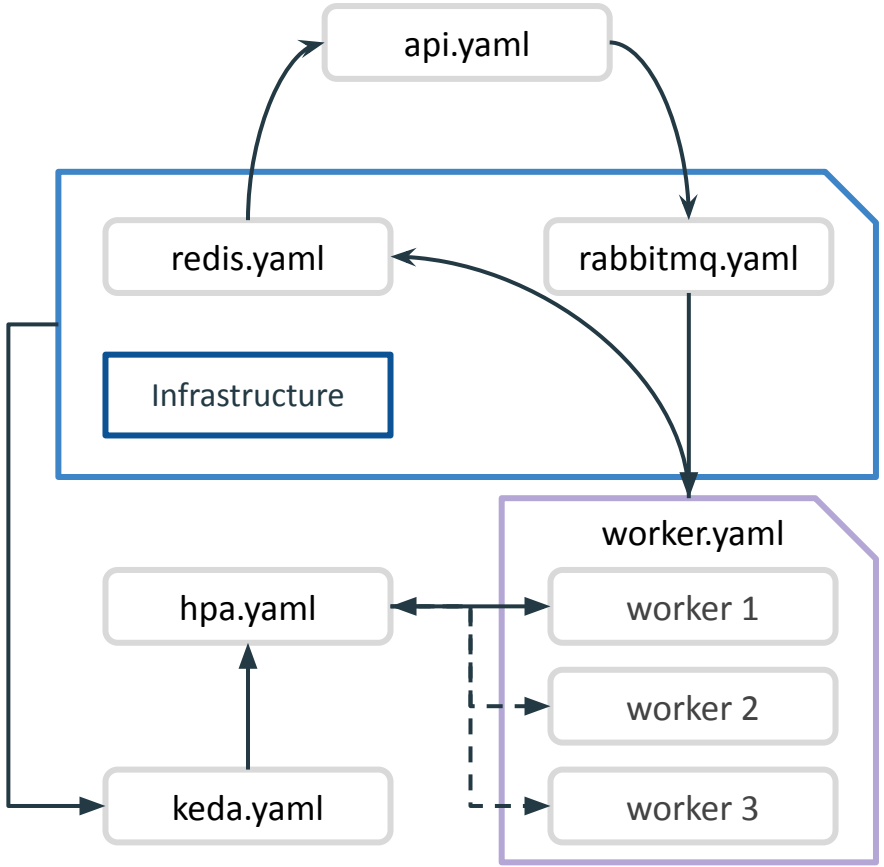
`Kubernetes` n'a pas de fonction build.

Il faut builder les images avec les `Dockerfiles` et les commandes `docker`.

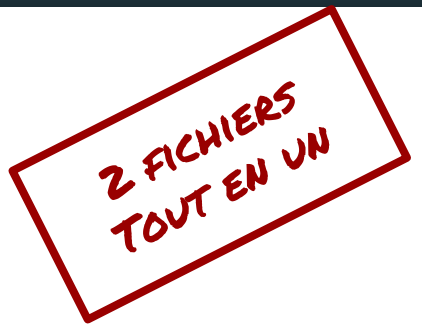
La raison pour laquel il y a un docker compose est pour valider toutes les images avant migration vers `kubernetes`.

`hpa.yaml` surveille la consommation des workers (cpu - ram), si il y a risque de dépassement alors il ouvre d'autres workers

Une extension (KEDA) permet de surveiller le nombre de job en attente (scaling sur le nombre)



Structure d’un yaml



Service	Déploiement
Rôle : L'Identité Réseau (Statique) Le Service crée une adresse IP et un nom fixe (ex: api-service) qui ne changent jamais. Il cherche des Pods ayant une étiquette spécifique (ex: app: worker) pour leur envoyer du trafic.	**Rôle** : Le Cœur Opérationnel (Dynamique) Le Déploiement définit l'image Docker à utiliser, les ressources (CPU/RAM) et le nombre de réplicas. Il contient le plan de fabrication des Pods. Si un Pod crash, le Déploiement en relance un nouveau.

Le Déploiement crée les instances (les Pods), mais comme ces instances peuvent mourir et changer d'adresse IP, on place un Service devant pour que le reste du système sache toujours comment leur parler.

Autres possibilités

```
infrastructure/
├── front-deployment.yaml
├── front-service.yaml
├── api-deployment.yaml
├── api-service.yaml
├── rabbitmq-deployment.yaml
├── rabbitmq-service.yaml
├── celery-worker-deployment.yaml
├── celery-hpa.yaml
├── redis-deployment.yaml
└── keda-scaledobject.yaml
```

```
k8s/
├── front/
│   ├── deployment.yaml
│   └── service.yaml
├── api/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── hpa.yaml
└── messaging/
    ├── rabbitmq.yaml
    └── celery-worker.yaml
```

Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector:
    app: api
  ports:
    - protocol: TCP
      port: 8000
      targetPort: 8000
```

`apiVersion` : propre à kubernetes (un peu comme les version des bibliothèques python) pour les `services` on prend `v1`.

`kind` : permet de savoir si c'est un service

`metadata.name` : le nom de la DNS interne (`http://api:8000`)

`spec.selector.app` : étiquette collé sur les réplicas du service (quand il s'éteint puis revient, ou en cas de duplication) ca permet de trouver les pods.

`ports` :

- **`protocol`** : TCP standard pour internet

  `port` : (`http://api:8000`)

  `targetPort` : celui du Dockerfile par exemple

Déploiement

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
```

`apiVersion` : propre à kubernetes pour les `Deployment` on prend `apps/v1`.

`kind` : permet de savoir si c'est un déploiement

`metadata.name` : le nom qui apparait par exemple en faisant `kubectl logs`

`spec.replicas` : Le nombre d'exemplaires du Pod à maintenir en vie. Si tu mets 3, Kubernetes lancera 3 copies de ton API.

`selector.matchLabels`: Le Deployment surveille uniquement les Pods qui ont le label `app: api`.

Il correspond aux `selector:app:api` du `service`

Déploiement

```
template:
  metadata:
    labels:
      app: api
  spec:
    containers:
    - name: api
      image: <>/<>:latest
      imagePullPolicy: IfNotPresent
      command: [""]
      env:
      - name: name
        value: "value"
```

`metadata.labels.app`: c'est grâce à ça que le Service (dont on a parlé avant) et le Deployment se comprennent

`spec.containers.name`: le nom du conteneur

`spec.containers.image`: l'image hébérgé sur dockerhub

`spec.containers.imagePullPolicy` : on télécharge si elle n'est pas en local dans kubernetes (et non docker).

`spec.containers.command`: La commande à exécuter (comme lancer uvicorn, celery ou streamlit).

`spec.containers.env`: Les variables d'environnements à donner au conteneur (**`name`** et **`value`**)

Construction et destruction des pods

Show 15 workers Search:

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@worker-deployment-687cbf866c-rd7zt	Online	0	0	0	5	0	0.23, 0.16, 0.09
celery@worker-deployment-687cbf866c-49qxv	Offline	0	0	0	4	0	N/A
celery@worker-deployment-687cbf866c-nkx2c	Offline	0	0	0	0	0	N/A
Total		**0**	**0**	**0**	**9**	**0**	

Showing 1 to 3 of 3 workers Previous 1 Next

`https://github.com/DeVerMyst/docker-compose-celery.git`